

SBA301 – SLOT 14

Chapter 11: Documenting REST Services

OpenAPI / Swagger Integration / springdoc + Documentation Demo

CLO3 – Describe RESTful APIs using Spring Boot built-in features and frameworks

Nền tảng: JDK 21 • Spring Boot 3 • Maven • IntelliJ IDEA • Browser • Postman

Phạm vi: Chapter 11, slide 1–37 | Không đi sâu JPA/Database

Mental model xuyên suốt: REST API đã chạy → mô tả bằng OpenAPI → springdoc sinh đặc tả → Swagger UI hiển thị → frontend/QA có thể đọc và thử API.

Quality rule: Documentation không chỉ “đẹp trên Swagger UI”. Nó phải khớp endpoint thật: method, path, parameter, request body, response schema, status code và error contract.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu được tổ chức theo đúng mạch học của các Slot 7–8–9–12–13: Prepare → Understand → Observe → Build → Break → Verify → Research → Transfer. Sinh viên nên đọc phần chuẩn bị trước buổi học, sau đó thực hiện Demo 1 trước khi chuyển sang Demo 2 và bài tập tổng hợp.

Pha học	Sinh viên làm gì?	Bảng chứng cần có
Prepare	Ôn REST, CRUD, annotations, 3-layer; kiểm tra JDK/Maven/IDE.	3-2-1 Learning Note + environment check
Understand	Phân biệt OpenAPI, Swagger, springdoc, Spring REST Docs.	Concept map hoặc bảng so sánh
Observe	Mở /v3/api-docs và Swagger UI, đọc schema/operations.	Ảnh chụp + giải thích
Build	Gắn annotations, cấu hình OpenAPI, chạy Demo 1/2.	Source + Swagger UI
Break	Cố tình sai version/path/annotation/schema.	Bug log
Verify	Đối chiếu Swagger với request/response thật.	Verification matrix
Research	Dùng prompt AI có kiểm chứng.	AI evidence log
Transfer	Document FUNewsManagementSystem hoặc resource khác.	API documentation package

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1 Kiến thức cần ôn lại từ Slot 13

- Spring Boot RESTful annotations: @RestController, mapping annotations, @PathVariable, @RequestBody, @RequestParam.
- Kiến trúc 3-layer: Controller → Service → Repository.
- CRUD và HTTP methods: GET, POST, PUT, DELETE.
- Request/Response handling, JSON serialization/deserialization, status code.
- Exception handling với @ExceptionHandler/@ControllerAdvice ở mức cơ bản.

0.2 Kiến thức nền tảng cần nhớ

- REST resource và endpoint là hai khái niệm khác nhau: resource là thực thể/ngữ nghĩa, endpoint là điểm truy cập cụ thể.
- API contract phải nói rõ method, URI, parameters, request body, response body, status code và error case.
- Maven dependency quyết định thư viện nào xuất hiện trên classpath.
- Annotation trong Java có thể cung cấp metadata cho framework hoặc công cụ sinh documentation.

0.3 Công cụ và kiểm tra môi trường

Công cụ	Mục đích	Tự kiểm tra
JDK 21	Compile/run Spring Boot	java -version
Maven	Dependency + build lifecycle	mvn -v
IntelliJ IDEA	IDE chính	Mở project, Maven sync không lỗi
Browser	Swagger UI	Mở localhost:8080
Postman	Đối chiếu behavior thật	Gửi GET tới API Slot 13
Spring Initializr	Tạo skeleton nếu bắt đầu project mới	Spring Web dependency

0.4 Pre-class 3-2-1 Learning Note

- 3 điều bạn nhớ về REST API và HTTP response.
- 2 câu hỏi: “API documentation nên chứa gì?” và “Swagger khác OpenAPI như thế nào?”.
- 1 dự đoán: nếu backend thay field JSON nhưng documentation không cập nhật, điều gì xảy ra với frontend?

0.5 Câu hỏi tự kiểm tra trước buổi học

1. REST API dùng để làm gì trong hệ thống React + Spring Boot?
2. Frontend developer cần biết những thông tin nào để gọi đúng một endpoint?
3. Status 200, 201, 400 và 404 truyền đạt điều gì?
4. Bạn có thể test API mà không viết frontend không?
5. Nếu controller đã chạy nhưng người dùng khác không biết request body, vấn đề nằm ở đâu?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Tóm tắt kiến thức Slot 13

Chủ đề	Nội dung chính	Ý nghĩa	Vấn đề Slot 14 giải quyết
REST fundamentals	Resource, URI, method, status, representation	Hiểu hợp đồng giao tiếp	Cần công bố hợp đồng đó cho người dùng API
Spring annotations	Mapping request vào controller	Viết API thực tế	Cần mô tả operation/parameter/response
3-layer CRUD	Controller–Service–Repository	API có cấu trúc	Cần tài liệu để tích hợp/kiểm thử
HTTP methods/status	GET/POST/PUT/DELETE + status code	Thể hiện semantics	Cần document success/error nhất quán

2. Định vị Slot 14 với các slot kế tiếp

Slot	Chủ đề	Mối liên hệ
12	REST Fundamentals	Xác định API contract ở mức khái niệm.
13	Annotations + 3-layer CRUD	Triển khai REST API.
14	Documenting REST Services	Biến code API thành tài liệu OpenAPI có thể đọc/test.
15	Versioning, Paging, Sorting	Documentation phải thể hiện version/query parameters/page model.
16	Testing, MockMvc + Lab 03	Tài liệu và tests cần khớp hành vi API.
17	JPA/Entity/Repository/Service	Schema thật phức tạp hơn; DTO/schema docs càng quan trọng.
18	REST Controller + Lab 04	Tích hợp backend hoàn thiện; Swagger hỗ trợ frontend.

3. Bối cảnh bài học

Ở Slot 13, sinh viên đã có thể viết CRUD REST API. Tuy nhiên “API chạy được” chưa đủ để làm việc nhóm. Frontend, mobile, QA và các service khác cần một hợp đồng có thể đọc, khám phá và kiểm thử. Chapter 11 giải quyết chính xác vấn đề này: biến API thành một interface được mô tả chuẩn hóa và có thể tương tác.

4. Mục tiêu bài học

1. Giải thích vai trò của API Documentation trong cộng tác và tích hợp.
2. Phân biệt OpenAPI Specification, Swagger toolchain, Swagger UI, springdoc-openapi và Spring REST Docs.
3. Thêm dependency springdoc-openapi phù hợp với Spring Boot 3.
4. Sử dụng @Tag, @Operation, @ApiResponse, @Parameter, OpenAPI @RequestBody và @Schema.
5. Customize OpenAPI metadata với @OpenAPIDefinition hoặc OpenAPI bean.
6. Truy cập /v3/api-docs và Swagger UI; dùng Try it out để gửi request.
7. Đối chiếu documentation với behavior thật của REST API.
8. Hoàn thành Demo 1 và Demo 2, sau đó chuyển giao sang FUNewsManagementSystem.

5. Khái niệm trung tâm

Khái niệm	Một câu cần nhớ
API Documentation	Mô tả cách dùng API: contract, examples, errors, auth, versioning.
OpenAPI Specification (OAS)	Chuẩn mô tả HTTP APIs bằng document machine-readable.
Swagger	Hệ sinh thái công cụ xoay quanh OpenAPI; không đồng nghĩa hoàn toàn với OpenAPI.
springdoc-openapi	Thư viện đọc Spring MVC/runtime metadata để sinh OpenAPI description và Swagger UI.
Swagger UI	Giao diện web đọc OpenAPI document và cho phép thử operation.
Spring REST Docs	Cách tiếp cận test-driven sinh snippets/documentation; khác springdoc.

6. Flow tổng quát

Flow: Slot 13 REST API → springdoc scans routes/models/annotations → OpenAPI JSON/YAML → Swagger UI → developer đọc + Try it out → đối chiếu request/response thật.

7. Tình huống dẫn nhập – FUNewsManagementSystem

Backend đã có `GET /api/news`, `POST /api/news`, `PUT /api/news/{id}` và `DELETE /api/news/{id}`. Frontend developer vẫn cần biết: field nào bắt buộc, `id` nằm ở path hay body, response success có schema gì, khi không tìm thấy trả 404 hay null, và lỗi validation có cấu trúc nào. Swagger UI chỉ hữu ích nếu những chi tiết này được document đúng.

8. Tư duy kiến trúc quan trọng

- Documentation là một phần của API contract, không phải “phần trang trí sau khi code xong”.
- Generated documentation chỉ đúng bằng mức độ đúng của code/annotations; auto-generation không tự sửa thiết kế API tệ.
- Documentation phải cover success path và failure path.
- Schema hiển thị trên Swagger cần phản ánh DTO/public contract, không nhất thiết phản ánh entity nội bộ.
- Swagger UI là client thử nghiệm thuận tiện, nhưng không thay thế automated tests.

9. Tài liệu tham khảo

- Slide Chapter 11, trang 1–37.

- springdoc-openapi documentation: <https://springdoc.org/>
- OpenAPI Specification: <https://spec.openapis.org/oas/latest.html>
- Swagger tools: <https://swagger.io/>
- Baeldung – Spring REST OpenAPI Documentation.
- Giáo trình Slot 12 và Slot 13 của môn SBA301.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. Tầm quan trọng của API Documentation

Giá trị	Nếu có documentation tốt	Nếu thiếu/document sai
Clarity	Biết endpoint, input/output, errors	Hỏi thủ công, đoán payload
Integration	Frontend tích hợp nhanh	Sai field/method/status
Testing	QA tạo case trực tiếp	Test coverage dựa vào truyền miệng
Evolution	Theo dõi version/change	Breaking change khó phát hiện
Onboarding	Developer mới tự học contract	Phụ thuộc người viết API

2. Spring REST Docs và springdoc-openapi – không phải một công cụ

Tiêu chí	Spring REST Docs	springdoc-openapi
Cách sinh tài liệu	Từ tests/snippets	Từ runtime mapping + annotations
Điểm mạnh	Documentation gắn với test đã chạy	Nhanh, tự sinh OpenAPI + Swagger UI
Output điển hình	Asciidoctor/HTML/PDF	OpenAPI JSON/YAML + Swagger UI
Phù hợp Slot 14	Khái niệm/so sánh	Công cụ thực hành chính

Không nhầm lẫn: Slide giới thiệu Spring REST Documents như một phương pháp test-driven. Phần demo của Slot 14 dùng springdoc-openapi để tạo OpenAPI/Swagger UI. Hai hướng này có thể cùng tồn tại trong dự án.

3. OpenAPI Specification và Swagger

- OpenAPI Specification (OAS) là format/chuẩn mô tả API: paths, operations, parameters, requestBodies, responses, schemas, security, servers...
- Swagger là họ công cụ như Swagger UI, Swagger Editor, Swagger Core; nhiều công cụ đọc/ghi OpenAPI.
- springdoc-openapi là thư viện tích hợp Spring Boot và OpenAPI, tự sinh `/v3/api-docs` và cung cấp Swagger UI khi thêm starter UI.

4. Cập nhật tương thích springdoc – Spring Boot

Nền tảng	Nhánh springdoc phù hợp	Ghi chú
Spring Boot 3.x	springdoc-openapi 2.x	Dùng `springdoc-openapi-starter-webmvc-ui` ; ưu tiên bản 2.x mới tương thích.
Spring Boot 4.x	springdoc-openapi 3.x	Nhánh 3.x được phát hành cho Spring Boot 4.

Slide cũ	Ví dụ 2.3.0	Dùng để hiểu cú pháp; khi lab nên pin phiên bản do giảng viên công bố.
----------	-------------	--

Course compatibility note: Trong tài liệu này, ví dụ Maven dùng `2.9.1` cho Spring Boot 3.x theo nhánh 2.x hiện hành tại thời điểm biên soạn. Nếu lớp khóa một BOM/version khác, ưu tiên version do giảng viên công bố.

Dependency mẫu cho Spring MVC / Spring Boot 3.x

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.9.1</version>
</dependency>
```

5. 5 bước tích hợp Swagger/OpenAPI với Spring Boot

1. Thêm `springdoc-openapi-starter-webmvc-ui` vào `pom.xml` và Maven reload.
2. Chạy app, kiểm tra OpenAPI document tại `/v3/api-docs` và Swagger UI.
3. Bổ sung annotations cho controller, operation, parameter, response và model.
4. Customize metadata: title, version, description, contact, license, server.
5. Dùng Swagger UI để thử operation; đối chiếu kết quả với Postman/API behavior.

6. Controller annotation – @Tag

@Tag nhóm các operations trong UI theo một domain/resource. Đặt tag ở controller giúp tài liệu dễ scan khi API lớn.

```
@Tag(
  name = "News",
  description = "CRUD operations for news articles"
)
@RestController
@RequestMapping("/api/news")
public class NewsController {
  // ...
}
```

Best practice: Tên tag nên phản ánh resource/domain, không phản ánh tên class kỹ thuật như "NewsControllerV2Impl".

7. Method annotation – @Operation

@Operation mô tả mục đích của một endpoint. `summary` nên ngắn để scan; `description` dùng cho rule, side effect, quyền hoặc lưu ý nghiệp vụ.

```
@Operation(
  summary = "Get all news",
  description = "Returns all news articles. Optional keyword filtering is supported."
)
@GetMapping
public List<News> getAll(@RequestParam(required = false) String keyword) {
  // ...
}
```

8. Response documentation – @ApiResponse / @ApiResponses

```
@ApiResponses({
  @ApiResponse(responseCode = "200", description = "News found"),
  @ApiResponse(responseCode = "404", description = "News not found"),
  @ApiResponse(responseCode = "500", description = "Unexpected server error")
})
```

```
@GetMapping("/{id}")
public News getById(@PathVariable Long id) {
    // ...
}
```

Một operation nên document các response mà client thực sự cần xử lý. Không nên liệt kê 10 status codes “cho đủ” nếu API không thể tạo ra chúng.

9. Parameter documentation – @Parameter

```
@GetMapping("/{id}")
public News getById(
    @Parameter(description = "News identifier", example = "10", required = true)
    @PathVariable Long id
) {
    // ...
}
```

Dùng `@Parameter` cho path/query/header parameter khi tên Java không đủ diễn đạt business meaning, range, example hoặc tính bắt buộc.

10. Request body documentation và lỗi import dễ gặp

Có hai annotation cùng tên `@RequestBody` : một của Spring (`org.springframework.web.bind.annotation.RequestBody`) để bind JSON runtime, và một của OpenAPI (`io.swagger.v3.oas.annotations.parameters.RequestBody`) để mô tả documentation. Khi cần dùng cả hai, hãy import có chủ đích hoặc dùng fully qualified name cho một annotation.

```
@Operation(
    summary = "Create news",
    requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
        required = true,
        description = "News payload"
    )
)
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public News create(
    @org.springframework.web.bind.annotation.RequestBody News news
) {
    // ...
}
```

Debug hint: Nếu IDE import nhầm `@RequestBody`, code có thể compile lỗi hoặc request runtime không bind JSON. Luôn kiểm tra package của annotation.

11. Model documentation – @Schema

```
@Schema(description = "News article exposed by the REST API")
public class News {

    @Schema(description = "Unique identifier", example = "10")
    private Long id;

    @Schema(description = "News title", example = "FPTU launches a new lab", requiredMode =
Schema.RequiredMode.REQUIRED)
    private String title;

    @Schema(description = "Publication status", example = "true")
    private boolean active;
}
```

Schema descriptions và examples giúp Swagger UI hiển thị model dễ hiểu hơn. Trong hệ thống thật, nên annotate DTO/public contract thay vì entity persistence nếu hai khái niệm khác nhau.

12. Customize OpenAPI definition

Có hai hướng phổ biến: dùng `@OpenAPIDefinition` cho metadata đơn giản, hoặc tạo `OpenAPI` bean để cấu hình linh hoạt hơn.

```
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI funewsOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("FUNews Management API")
                .version("v1")
                .description("REST API for News, Category and User management")
                .contact(new Contact().name("SBA301 Team")))
            .addServersItem(new Server()
                .url("http://localhost:8080")
                .description("Local development"));
    }
}
```

13. Swagger UI và OpenAPI endpoints

URL	Vai trò
http://localhost:8080/swagger-ui.html	Entry URL thường dùng; có thể redirect đến UI path.
http://localhost:8080/swagger-ui/index.html	UI path thường thấy trong nhiều phiên bản/configuration.
http://localhost:8080/v3/api-docs	OpenAPI JSON machine-readable.
http://localhost:8080/v3/api-docs.yaml	OpenAPI YAML nếu bật/được hỗ trợ.

14. Try it out – đọc trước khi bấm Execute

1. Xác định operation và endpoint.
2. Đọc mô tả parameter/request body.
3. Mở schema và examples.
4. Dự đoán status code + response trước khi Execute.
5. Execute và đọc request URL/curl, response status, headers, body.
6. Nếu kết quả khác documentation, ghi mismatch thay vì sửa documentation theo kiểu “che lỗi”.

15. SecurityScheme awareness

Nếu API có bearer token/JWT, OpenAPI có thể định nghĩa security scheme. Slot này chỉ giới thiệu ở mức awareness; Spring Security chi tiết thuộc phần khác.

```
@SecurityScheme(
    name = "bearerAuth",
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT"
)
@Configuration
public class OpenApiSecurityConfig {
}
```

16. Những hiểu sai phổ biến

Hiểu sai	Điều chỉnh
----------	------------

Swagger = OpenAPI	OpenAPI là specification; Swagger là hệ sinh thái công cụ.
Thêm dependency là document xong	Auto-generated metadata thường chưa đủ business meaning/error contract.
Swagger UI thay thế tests	UI giúp exploration/manual verification; tests vẫn cần.
@Schema càng nhiều càng tốt	Chỉ annotate thông tin tăng giá trị, tránh mô tả hiển nhiên.
Documentation luôn đúng vì tự sinh	Nếu code annotations hoặc response thực tế sai, docs cũng sai.
Swagger UI nên public ở production	Tùy chính sách bảo mật; nhiều hệ thống hạn chế hoặc bảo vệ docs.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

1. Ai sử dụng API documentation?

Vai trò	Cần gì từ documentation?
Frontend developer	Endpoint, method, params, payload, schemas, error codes.
Mobile developer	Contract giống frontend nhưng chú ý network/errors/version.
QA	Danh sách operations + expected responses để thiết kế test.
Backend developer khác	Integration giữa services.
Tech lead/architect	Consistency, versioning, security, evolution.

2. Case study – FUNewsManagementSystem

Resource	Operations cần document	Điểm dễ mismatch
News	CRUD + search/filter	categoryId, status, tags, 404, validation
Category	CRUD + active/inactive	name uniqueness, status semantics
User/Account	CRUD + role	Không expose password; role code vs label

3. Documentation contract cho một endpoint

Thuộc tính	Ví dụ GET /api/news/{id}
Summary	Get news by id
Path parameter	id: Long, required, example 10
Success	200 + News schema
Not found	404 + ApiError schema
Security	Nếu bảo vệ: bearerAuth
Example	News JSON hợp lệ

4. Documentation drift – lỗi sản phẩm thật

Documentation drift xảy ra khi code và tài liệu khác nhau. Ví dụ Swagger nói `categoryId` bắt buộc nhưng backend lại chấp nhận null; hoặc Swagger nói DELETE trả 204 nhưng controller trả 200 + body. Sinh viên phải học cách phát hiện drift bằng verification matrix.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN THỰC HIỆN VÀ TRIỂN KHAI

A. Demo 1 – Simple Documenting REST API

A1. Mục tiêu

- Tạo Spring Boot project có Spring Web.
- Thêm springdoc starter.
- Tạo Employee model + endpoint đọc dữ liệu.
- Quan sát OpenAPI JSON và Swagger UI.
- Bổ sung @Tag, @Operation, @ApiResponse, @Schema.

A2. Tạo project

Thiết lập	Giá trị gợi ý
Project	Maven
Language	Java
Java	21
Spring Boot	3.x theo course
Dependencies	Spring Web, Lombok (optional)
Artifact	s14-openapi-demo

A3. pom.xml – thêm springdoc

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.9.1</version>
</dependency>
```

A4. Employee model

```
@Schema(description = "Employee exposed by Demo 1")
public class Employee {
    @Schema(example = "1")
    private Long id;

    @Schema(example = "Nguyen Van A")
    private String name;

    @Schema(example = "Backend Developer")
    private String position;

    // constructors, getters, setters
}
```

A5. EmployeeController

```
@Tag(name = "Employees", description = "Simple documentation demo")
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final List<Employee> data = List.of(
```

```

        new Employee(1L, "Nguyen Van A", "Backend Developer"),
        new Employee(2L, "Tran Thi B", "QA Engineer")
    );

    @Operation(summary = "Get all employees")
    @ApiResponse(responseCode = "200", description = "Employees returned")
    @GetMapping
    public List<Employee> getAll() {
        return data;
    }

    @Operation(summary = "Get employee by id")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Employee found"),
        @ApiResponse(responseCode = "404", description = "Employee not found")
    })
    @GetMapping("/{id}")
    public ResponseEntity<Employee> getById(
        @Parameter(description = "Employee id", example = "1")
        @PathVariable Long id
    ) {
        return data.stream()
            .filter(e -> e.getId().equals(id))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}

```

A6. Run & verify

1. Run application.
2. Mở ` /v3/api-docs` ; tìm path ` /api/employees` .
3. Mở Swagger UI; kiểm tra tag Employees.
4. Dùng Try it out cho GET list và GET by id.
5. Thử id không tồn tại; xác nhận response 404 khớp documentation.

Checkpoint V1: Sinh viên chụp 3 bằng chứng: Swagger UI overview, operation detail, request/response đã Execute.

B. Demo 2 – Customize REST with Swagger trên 3-layer

B1. Cấu trúc thư mục

```

src/main/java/.../
├── config/
│   └── OpenApiConfig.java
├── controller/
│   └── NewsController.java
├── service/
│   └── NewsService.java
├── repository/
│   └── NewsRepository.java
├── model/
│   └── News.java
├── exception/
│   ├── NewsNotFoundException.java
│   ├── ApiError.java
│   └── GlobalExceptionHandler.java

```

B2. Model – News

```

@Schema(description = "News article")
public class News {
    @Schema(example = "10", accessMode = Schema.AccessMode.READ_ONLY)
    private Long id;
}

```

```

@Schema(example = "SBA301 Slot 14", requiredMode = Schema.RequiredMode.REQUIRED)
private String title;

@Schema(example = "OpenAPI documentation practice")
private String content;

@Schema(example = "1")
private Long categoryId;

@Schema(example = "true")
private boolean active;

// constructors, getters, setters
}

```

B3. Repository in-memory

```

@Repository
public class NewsRepository {
    private final Map<Long, News> store = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong(0);

    public List<News> findAll() {
        return new ArrayList<>(store.values());
    }

    public Optional<News> findById(Long id) {
        return Optional.ofNullable(store.get(id));
    }

    public News save(News news) {
        if (news.getId() == null) {
            news.setId(sequence.incrementAndGet());
        }
        store.put(news.getId(), news);
        return news;
    }

    public void deleteById(Long id) {
        store.remove(id);
    }
}

```

B4. Service

```

@Service
public class NewsService {
    private final NewsRepository repository;

    public NewsService(NewsRepository repository) {
        this.repository = repository;
    }

    public List<News> findAll() {
        return repository.findAll();
    }

    public News findById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new NewsNotFoundException(id));
    }

    public News create(News news) {
        news.setId(null);
        return repository.save(news);
    }

    public News update(Long id, News input) {
        News current = findById(id);
        current.setTitle(input.getTitle());
        current.setContent(input.getContent());
        current.setCategoryId(input.getCategoryId());
        current.setActive(input.isActive());
        return repository.save(current);
    }
}

```

```

    public void delete(Long id) {
        findById(id);
        repository.deleteById(id);
    }
}

```

B5. Error contract

```

@Schema(description = "Standard API error")
public record ApiError(
    @Schema(example = "404") int status,
    @Schema(example = "News 99 not found") String message,
    @Schema(example = "/api/news/99") String path
) {}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NewsNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(
        NewsNotFoundException ex,
        HttpServletRequest request
    ) {
        ApiError error = new ApiError(
            404, ex.getMessage(), request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
}

```

B6. Controller được document

```

@Tag(name = "News", description = "News management operations")
@RestController
@RequestMapping("/api/news")
public class NewsController {
    private final NewsService service;

    public NewsController(NewsService service) {
        this.service = service;
    }

    @Operation(summary = "Get all news")
    @ApiResponse(responseCode = "200", description = "News list returned")
    @GetMapping
    public List<News> getAll() {
        return service.findAll();
    }

    @Operation(summary = "Get news by id")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "News found"),
        @ApiResponse(responseCode = "404", description = "News not found")
    })
    @GetMapping("/{id}")
    public News getById(
        @Parameter(description = "News id", example = "1")
        @PathVariable Long id
    ) {
        return service.findById(id);
    }

    @Operation(summary = "Create news")
    @ApiResponse(responseCode = "201", description = "News created")
    @PostMapping
    public ResponseEntity<News> create(
        @org.springframework.web.bind.annotation.RequestBody News input
    ) {
        News created = service.create(input);
        URI location = URI.create("/api/news/" + created.getId());
        return ResponseEntity.created(location).body(created);
    }

    @Operation(summary = "Update news")

```

```

@ApiResponses({
    @ApiResponse(responseCode = "200", description = "News updated"),
    @ApiResponse(responseCode = "404", description = "News not found")
})
@PutMapping("/{id}")
public News update(
    @PathVariable Long id,
    @org.springframework.web.bind.annotation.RequestBody News input
) {
    return service.update(id, input);
}

@Operation(summary = "Delete news")
@ApiResponses({
    @ApiResponse(responseCode = "204", description = "News deleted"),
    @ApiResponse(responseCode = "404", description = "News not found")
})
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
}

```

B7. Customize OpenAPI metadata

```

@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI funewsOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("FUNews Management API")
                .version("1.0.0")
                .description("Training API for SBA301 Slot 14")
                .termsOfService("Training use only")
                .contact(new Contact()
                    .name("SBA301 Team")
                    .email("team@example.edu")))
            .addServersItem(new Server()
                .url("http://localhost:8080")
                .description("Local development"));
    }
}

```

B8. Verification matrix

Case	Swagger mô tả	Thực thi	Expected
GET list	200 + array News	Execute GET /api/news	200, JSON array
GET id	200/404	id có/không tồn tại	Khớp 200 hoặc 404
POST	201 + News	Body hợp lệ	201, Location header, JSON body
PUT	200/404	Cập nhật id	200 hoặc 404
DELETE	204/404	Xóa id	204 hoặc 404

B9. Break-It Lab

1. Xóa dependency springdoc → Swagger UI còn truy cập được không?
2. Import nhầm OpenAPI @RequestBody thay cho Spring @RequestBody → runtime thay đổi thế nào?
3. Document 201 nhưng controller trả 200 → xác định documentation drift.
4. Đổi server.port=8081 nhưng OpenAPI server vẫn 8080 → Try it out có vấn đề gì?
5. Đổi field `categoryId` thành `category` trong code nhưng giữ mô tả cũ → schema lệch ra sao?

6. Document 404 nhưng service trả null/200 → frontend sẽ bị ảnh hưởng thế nào?

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Lỗi thường gặp khi cài springdoc

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
404 Swagger UI	Sai URL / starter chưa có / app chưa chạy	Kiểm tra `/v3/api-docs`, dependency, port.
Maven resolve lỗi	Version không tương thích hoặc chưa reload	Kiểm tra Boot major ↔ springdoc major/minor.
Endpoint không xuất hiện	Controller package không được scan hoặc mapping khác	Kiểm tra package, Spring mapping, app logs.
Schema thiếu field	Accessor/visibility/annotation/Jackson configuration	Kiểm tra model/DTO và generated `/v3/api-docs`.
Try it out gọi sai host	Server metadata/config sai	Sửa `servers` hoặc bỏ hard-code local URL.

2. Best practices cho annotation

- @Operation summary ngắn, description chỉ thêm điều không hiển nhiên từ method/path.
- @ApiResponse tập trung các response client cần xử lý; document error schema nếu có chuẩn chung.
- @Schema dùng examples thực tế nhưng không lộ secret/password/token.
- Không copy cùng một description vào mọi endpoint.
- Ưu tiên DTO/public model nếu entity chứa field nội bộ.
- Giữ documentation gần code nhưng vẫn có API design review độc lập.

3. Documentation quality gate

Gate	PASS khi...
G1 Discoverability	Frontend tìm được operation theo tag/resource.
G2 Request contract	Path/query/body và required fields rõ.
G3 Response contract	Success + error status/schema rõ.
G4 Executability	Try it out chạy được hoặc giới hạn được giải thích.
G5 Accuracy	Swagger khớp behavior thật qua manual verification.
G6 Security hygiene	Không expose secrets/example credentials thật.

4. Debug decision tree

Swagger không mở? App chạy? → `/v3/api-docs` mở? → dependency đúng? → Boot/springdoc compatible? → path/port/context-path đúng? → security có chặn docs không?

Docs sai? Kiểm tra mapping runtime → annotation OpenAPI → model/schema/Jackson → global error handler → actual response bằng Postman/Swagger Execute.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

A. Bài tập chủ đề

Bài	Yêu cầu
Bài 1 – Dependency & UI	Thêm springdoc starter, chạy app, chụp ` /v3/api-docs` và Swagger UI.
Bài 2 – @Tag	Tạo ProductController và nhóm operations dưới tag Products.
Bài 3 – @Operation	Viết summary/description cho GET list, GET detail, POST.
Bài 4 – @ApiResponse	Document 200/201/400/404/500 có chọn lọc.
Bài 5 – @Parameter	Document path id và query keyword/category.
Bài 6 – @Schema	Annotate Product/News DTO với description + example.
Bài 7 – OpenAPI config	Customize title/version/description/server.
Bài 8 – Drift detection	Cố tình làm code và docs khác nhau; ghi mismatch.
Bài 9 – UI vs Postman	Gửi cùng một request bằng Swagger UI và Postman; so sánh.
Bài 10 – Documentation review	Đổi vai frontend reviewer và ghi 5 câu hỏi còn thiếu trong docs.

B. Bài tập tổng hợp – Simple Documenting REST API for FUNewsManagementSystem

1. Dùng CRUD News in-memory theo kiến trúc 3-layer.
2. Thêm springdoc-openapi và Swagger UI.
3. Document Controller bằng @Tag.
4. Document CRUD methods bằng @Operation và @ApiResponse.
5. Document path/query parameters bằng @Parameter.
6. Document News/ApiError schema bằng @Schema.
7. Customize OpenAPI info và server.
8. Test toàn bộ CRUD từ Swagger UI.
9. Đối chiếu bằng Postman ít nhất 3 cases.
10. Nộp Evidence Package và giải thích một documentation mismatch đã phát hiện/sửa.

Rubric tự đánh giá mini – 100 điểm

Tiêu chí	Điểm
Project + Swagger UI chạy	15
Endpoint discovery đầy đủ	10
Operation/parameter docs	20
Response/error docs	15
Schema docs	10
OpenAPI customization	10
Try it out + verification	10

Code/3-layer consistency	5
Evidence/README	5

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
API Documentation	Tài liệu mô tả cách dùng API.	Endpoint table / Swagger UI
OpenAPI Specification	Chuẩn machine-readable mô tả HTTP API.	/v3/api-docs
Swagger	Hệ sinh thái công cụ cho OpenAPI.	Swagger UI
springdoc-openapi	Thư viện tích hợp Spring Boot với OpenAPI.	starter-webmvc-ui
Swagger UI	UI tương tác với OpenAPI document.	Try it out
Spring REST Docs	Toolchain test-driven tạo documentation snippets.	MockMvc + AsciiDoctor
@Tag	Nhóm operations theo domain/resource.	News
@Operation	Mô tả một API operation.	summary/description
@ApiResponse	Mô tả một response theo status.	404 Not found
@Parameter	Mô tả parameter.	path id / query keyword
OpenAPI @RequestBody	Mô tả request body trong spec.	News payload
Spring @RequestBody	Bind HTTP body thành Java object.	JSON → News
@Schema	Mô tả model/property schema.	News.title
@OpenAPIDefinition	Metadata cấp ứng dụng cho OpenAPI.	Info/servers
@SecurityScheme	Mô tả scheme xác thực.	Bearer JWT
@Configuration	Đánh dấu class cấu hình Spring.	OpenApiConfig
@Bean	Đăng ký object vào Spring container.	OpenAPI bean
OpenAPI object	Root Java object của OpenAPI model.	new OpenAPI()
Info	Metadata API như title/version/contact.	FUNews API v1
Server	Base URL mô tả nơi API chạy.	localhost:8080
3-layer	Controller–Service–Repository.	News flow
Endpoint	Method + path có thể gọi.	GET /api/news
Request	Dữ liệu client gửi.	headers/body/params
Response	Kết quả server trả.	status/headers/body
Schema	Cấu trúc dữ liệu trong contract.	News / ApiError
Documentation drift	Docs và behavior thật không đồng bộ.	Docs 201 nhưng API 200
Try it out	Chế độ gửi request từ Swagger UI.	Execute
OpenAPI JSON	Spec sinh ở dạng JSON.	/v3/api-docs

VIII. RESEARCH BASE VÀ AI BASE LEARNING

AI rule: AI may generate; student must verify. Mọi câu trả lời AI về version, annotation hoặc security phải được kiểm tra lại bằng official docs hoặc bằng behavior của project.

A. 10 prompt tự nghiên cứu chuyên sâu kỹ thuật

1. Phân tích sự khác nhau giữa OpenAPI 2.0/Swagger 2.0 và OpenAPI 3.x.
2. Giải thích cách springdoc-openapi khám phá Spring MVC mappings và model schemas.
3. Phân tích cách @Schema ảnh hưởng đến generated schema và Swagger UI.
4. Giải thích khác nhau giữa @Tag và @Operation; ví dụ misuse.
5. Phân tích cách @ApiResponse kết hợp Content/Schema để document error body.
6. Giải thích hai cách customize OpenAPI: @OpenAPIDefinition và OpenAPI bean.
7. Phân tích cách Swagger UI đọc /v3/api-docs và render operations.
8. Giải thích security schemes trong OpenAPI 3 và cách Swagger UI gửi bearer token.
9. Phân tích reusable components/schemas trong OpenAPI 3.
10. Giải thích giới hạn của annotation-driven documentation so với test-driven documentation.

B. 10 prompt nghiên cứu thực tiễn production

1. Trong dự án thực tế, team duy trì API documentation đồng bộ với code bằng quy trình nào?
2. Làm thế nào review OpenAPI contract trong pull request?
3. Trong microservices, nên có Swagger UI riêng từng service hay API portal tập trung?
4. Làm thế nào document authentication/authorization mà không lộ secret?
5. Có nên expose Swagger UI ở production? Phân tích theo threat model.
6. Làm thế nào version API documentation và phát hiện breaking changes?
7. Ai chịu trách nhiệm chất lượng API documentation trong team cross-functional?
8. Document pagination/sorting/filtering thế nào để client hiểu semantics?
9. CI/CD có thể validate OpenAPI document bằng những quality gates nào?
10. Làm thế nào mô tả rate limiting, error format và correlation id trong API docs?

C. 10 prompt cập nhật công nghệ và xu hướng

1. So sánh springdoc-openapi với springfox và lý do migration trong Spring Boot 3.
2. Swagger Editor hỗ trợ design-first workflow như thế nào?
3. OpenAPI Generator có thể sinh client SDK/server stub ra sao?
4. Redoc/Redocly khác Swagger UI về trải nghiệm documentation thế nào?
5. Stoplight hỗ trợ API design/governance thế nào?
6. Postman và Swagger UI bổ sung nhau hay thay thế nhau?
7. GraphQL schema/documentation khác OpenAPI REST như thế nào?
8. gRPC/Protobuf documentation khác REST/OpenAPI như thế nào?
9. AsyncAPI giải quyết loại API nào mà OpenAPI không tập trung?
10. AI có thể hỗ trợ phát hiện documentation drift hoặc sinh examples/test cases như thế nào?

D. AI Verification Worksheet

Prompt	Claim của AI	Cách kiểm chứng	Kết quả	Điều chỉnh
...	...	Official docs / run project / inspect /v3/api-docs	Pass/Fail	...
...	...	Swagger UI vs Postman	Pass/Fail	...

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Knowledge Check

1. API Documentation là gì và ai là người dùng?
2. OpenAPI Specification khác Swagger như thế nào?
3. springdoc-openapi làm gì trong Spring Boot?
4. Spring REST Docs khác springdoc-openapi ở cách sinh documentation như thế nào?
5. @Tag dùng ở cấp nào? @Operation dùng ở cấp nào?
6. @ApiResponse nên document gì?
7. @Parameter phù hợp với loại input nào?
8. Vì sao OpenAPI @RequestBody dễ bị import nhầm với Spring @RequestBody?
9. @Schema giúp gì cho frontend/QA?
10. OpenAPI bean có thể customize những metadata nào?
11. URL nào chứa OpenAPI JSON?
12. Swagger UI có thay thế Postman và automated tests không? Vì sao?

2. Skill Check

Kỹ năng	Đạt	Chưa đạt	Cần làm lại
Thêm springdoc-openapi dependency	☐	☐	☐
Mở /v3/api-docs	☐	☐	☐
Mở Swagger UI	☐	☐	☐
Thêm @Tag/@Operation	☐	☐	☐
Document response và parameter	☐	☐	☐
Document model bằng @Schema	☐	☐	☐
Customize OpenAPI info	☐	☐	☐
Execute CRUD từ Swagger UI	☐	☐	☐
Phát hiện documentation drift	☐	☐	☐

3. Mức tự đánh giá

Mức	Dấu hiệu
-----	----------

Mức 1 – Chưa đạt	Không mở được Swagger UI; chưa phân biệt OpenAPI/Swagger; annotation dùng theo mẫu nhưng không hiểu.
Mức 2 – Đạt cơ bản	Tích hợp springdoc; document operations/schemas; dùng Try it out.
Mức 3 – Tốt	Thiết kế docs rõ ràng; error contract đầy đủ; customize metadata; phát hiện mismatch; chuyển giao sang resource khác.

4. Human Verification Gate

1. Mở generated `/v3/api-docs` và chỉ ra `paths` của News.
2. Giải thích vì sao @RestController không phải annotation OpenAPI.
3. Chỉ ra package của Spring @RequestBody và OpenAPI @RequestBody.
4. Sửa summary của một operation và chứng minh UI cập nhật.
5. Thêm 404 response cho một endpoint và giải thích error contract.
6. Đổi example trong @Schema và xác định vị trí thay đổi trên Swagger UI.
7. Giải thích flow từ controller annotations đến OpenAPI JSON rồi đến Swagger UI.
8. Dùng Try it out tạo News, sau đó đọc Location header.
9. Cố ý làm docs sai status, phát hiện mismatch bằng execution.
10. Giải thích khi nào nên dùng Spring REST Docs thay hoặc bổ sung springdoc.

5. Exit Ticket

- Một câu bạn có thể dùng để phân biệt OpenAPI và Swagger.
- Một annotation bạn thấy hữu ích nhất và vì sao.
- Một mismatch documentation/code bạn đã từng hoặc có thể gặp.
- Một câu hỏi muốn mang sang Slot 15 về versioning/paging/sorting.

PHỤ LỤC A – KẾ HOẠCH BUỔI HỌC 135 PHÚT

Thời gian	Hoạt động	Kết quả
0–10	Retrieval Slot 13 + 3-2-1	Kích hoạt kiến thức nền
10–25	Concept: docs/OpenAPI/Swagger/REST Docs	Phân biệt công cụ/chuẩn
25–40	springdoc + generated docs	Mở `/v3/api-docs` /Swagger UI
40–60	Annotations walkthrough	Hiểu Tag/Operation/Response/Schema
60–80	Demo 1	Simple documenting API chạy
80–105	Demo 2 3-layer + OpenAPI config	CRUD docs + Try it out
105–120	Break-It + documentation drift	Debug & verification
120–130	Mini exercise FUNews	Transfer
130–135	Exit ticket	Self-check + bridge Slot 15

PHỤ LỤC B – QUICK REFERENCE

Mục tiêu	Công cụ/annotation
Nhóm controller	@Tag
Mô tả operation	@Operation
Mô tả response	@ApiResponse / @ApiResponses
Mô tả path/query/header parameter	@Parameter
Mô tả request body trong OpenAPI	io.swagger.v3...parameters.RequestBody
Bind JSON runtime	org.springframework.web.bind.annotation.RequestBody
Mô tả model/property	@Schema
Metadata toàn API	@OpenAPIDefinition hoặc OpenAPI @Bean
Security awareness	@SecurityScheme
Spec JSON	/v3/api-docs
Swagger UI	/swagger-ui.html hoặc /swagger-ui/index.html

PHỤ LỤC C – EVIDENCE PACKAGE

Mã	Bảng chứng
E1	Environment check: Java/Maven/Spring Boot project
E2	pom.xml có springdoc starter
E3	OpenAPI JSON ` /v3/api-docs `
E4	Swagger UI overview
E5	Operation có @Operation/@ApiResponse
E6	Model schema có @Schema
E7	OpenAPI custom title/version/server
E8	Try it out success case
E9	Try it out error case
E10	Documentation drift / Break-It log
E11	AI Verification Worksheet nếu dùng AI
E12	Reflection + Exit ticket

Bridge sang Slot 15: Khi API có versioning, paging, sorting và filtering, documentation phải mô tả rõ query parameters, response page model và backward compatibility. Đây là bước kế tiếp sau khi sinh viên đã document CRUD cơ bản.